Ministerul Educaţiei și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 2

Study and Empirical Analysis of Sorting Algorithms

Elaborated:
st. gr. FAF-242 Bogdan Arama

Verified:
asist. univ. Fiștic Cristofor

Chișinău – 2026

Summary

ALGORITHM ANALYSIS 3
Objective 3
Tasks 3
Theoretical Notes 3
Introduction 4
Comparison Metric 4
Input Format 4
SORTING ALGORITHMS REPRESENTATION 5
QuickSort Lomuto 5
QuickSort Hoare 6
Merge Sort 7
Merge Sort Improved 8
Heap Sort 9
Heap Sort Floyd's Bottom-Up 10
Persian Sort 11
IMPLEMENTATION AND ANALYSIS 13
CONCLUSION 14

ALGORITHM ANALYSIS

Objective

The objective of this laboratory work is to study and analyze a selection of sorting algorithms, examining their implementations, the optimizations that distinguish improved variants from their standard counterparts, and their practical efficiency as measured through empirical testing and direct comparison.

Tasks

1. Implement seven sorting algorithms, including standard versions and optimized variations;
2. Document the working principles and internal logic of each implemented algorithm;
3. Explain the mechanical reasoning behind the improvements in the optimized variants;
4. Establish the properties of the input data distributions used for analysis;
5. Decide the comparison metrics used to evaluate algorithmic performance;
6. Analyze empirically the algorithms across the chosen datasets;
7. Present the results of the obtained data through graphical visualizations;
8. Deduce conclusions based on the observed hardware and software interactions.

Theoretical Notes

Sorting is one of the most fundamental operations in computer science, and its analysis has long been a central topic in the field. While mathematical complexity analysis provides theoretical bounds, empirical analysis offers a complementary perspective that is often more useful in practice. Empirical testing can reveal the complexity class of an algorithm from observed behavior, enable meaningful comparisons between multiple algorithms solving the same problem, and expose how a particular machine or runtime environment affects performance.

The general procedure followed in this study involves establishing a goal, choosing a metric for efficiency, defining input characteristics, implementing the algorithms in Python, and finally interpreting the collected data to find hardware-level bottlenecks.

Introduction

Sorting algorithms are essential tools for organizing data, forming the backbone of countless software systems. Different algorithms carry distinct performance profiles depending on the input size, the initial ordering of elements, and available computational resources. Understanding these trade-offs is not merely academic; it informs which algorithm is appropriate for a given real-world situation, where factors like CPU cache locality and branch prediction accuracy play a decisive role.

Comparison Metric

The comparison metric for this laboratory work is the execution time $T(n)$ of each algorithm. Execution time was chosen because it reflects the overall practical cost of running an algorithm, incorporating not just the number of comparisons but also memory accesses, cache behavior, and the overhead of the specific implementation.

Input Format

The algorithms were tested on datasets of nine different sizes: 100, 1,000, 10,000, 100,000, 200,000, 400,000, 600,000, 800,000, and 1,000,000 elements. To ensure results capture a broad picture of algorithmic behavior, five distribution patterns were used:

- **Unsorted:** Randomly ordered elements with no inherent structure.
- **Nearly Sorted (Organ):** Symmetric "hill" patterns rising to a peak and then mirroring down.
- **Sorted:** Arrays arranged in strictly ascending order.
- **Sorted Descending:** Arrays arranged in strictly reverse order.
- **Duplicate:** Arrays containing many repeated values to test partitioning resilience.

SORTING ALGORITHMS REPRESENTATION

1. QuickSort Lomuto

Overview

QuickSort Lomuto is a classic divide-and-conquer algorithm that relies on a specific partitioning scheme to organize data. It is often the first version of QuickSort taught due to its straightforward logic and easy implementation. However, it is structurally sensitive to the initial order of data, which can lead to performance bottlenecks in specific scenarios, such as pre-sorted arrays.

How It Works

The Lomuto scheme designates the last element of the array as the "pivot." The algorithm maintains a boundary index, typically i, which separates elements smaller than the pivot from those larger than it. As the algorithm iterates through the array with a second pointer, any element found to be less than the pivot is swapped into the "smaller" region, and the boundary i is moved forward. After the full scan, the pivot is swapped into its final position at the boundary. The algorithm then recursively applies this same logic to the sub-arrays on the left and right of the pivot.

```python
def partition(array, low, high):
    pivot = array[high]
    i = low - 1
    for j in range(low, high):
        if array[j] < pivot:
            i += 1
            array[i], array[j] = array[j], array[i]
    array[i + 1], array[high] = array[high], array[i + 1]
    return i + 1

def quicksort(array, low, high):
    if low < high:
        pi = partition(array, low, high)
        quicksort(array, low, pi - 1)
        quicksort(array, pi + 1, high)
```

Figure 1: QuickSort Lomuto Implementation

Complexity

- **Average Case:** $O(n \log n)$, where the pivot consistently splits the array into relatively equal halves.

- **Worst Case:** $O(n^2)$, occurring when the pivot is the smallest or largest element (common in sorted data).
- **Space Complexity:** $O(\log n)$ due to the depth of the recursive call stack.

2. QuickSort Hoare

Overview

The Hoare variant is an optimization of the original QuickSort logic developed by C.A.R. Hoare. It is generally more efficient than Lomuto because it performs significantly fewer swaps on average. By utilizing a two-pointer approach, it manages to partition the data by looking at it from both ends simultaneously, making it more resilient to various data distributions.

How It Works

Unlike Lomuto, the Hoare scheme uses two pointers that start at the beginning and the end of the array. These pointers move toward each other until they find a pair of elements that are on the "wrong side" of the pivot (an element larger than the pivot on the left, and a smaller one on the right). Once found, these two elements are swapped. This process continues until the pointers cross. This method is highly efficient because it only moves elements that strictly need to be moved, and it tends to produce more balanced partitions even when the data contains many duplicate values.

```python
def partition(array, low, high):
    pivot = array[low + (high-low)//2]
    i = low - 1
    j = high + 1
    while True:
        i += 1
        while array[i] < pivot: i += 1
        j -= 1
        while array[j] > pivot: j -= 1
        if i >= j: return j
        array[i], array[j] = array[j], array[i]

def quicksort(array, low, high):
    if low < high:
        pi = partition(array, low, high)
        quicksort(array, low, pi)
        quicksort(array, pi + 1, high)
```

Figure 2: QuickSort Hoare Implementation

Complexity

- **Average Case:** $O(n \log n)$, typically with a smaller constant factor than Lomuto.
- **Worst Case:** $O(n^2)$, though this is much harder to trigger in practice than with Lomuto.
- **Space Complexity:** $O(\log n)$ for recursive stack management.

3. Merge Sort

Overview

Merge Sort is a stable, comparison-based sorting algorithm that embodies the "divide and conquer" paradigm. Its primary strength is its guaranteed performance; unlike QuickSort, its speed is not dependent on the initial distribution of the data. It is also a stable sort, meaning it preserves the relative order of elements with equal keys.

How It Works

The algorithm recursively divides the array into two equal halves until it reaches base cases consisting of single-element arrays. It then begins the "Merge" phase, where it compares the smallest remaining elements of two sorted sub-arrays and places the smaller one into a temporary auxiliary array. This process continues until all elements are merged back

into a single sorted structure. Because the merging happens in a controlled, sequential manner, the algorithm consistently avoids the $O(n^2)$ traps that affect QuickSort.

```
def merge(array,l,m,r):
    n1 = m-l+1
    n2 = r-m

    L = [0]*n1
    R = [0]*n2

    for i in range(n1):
        L[i]=array[l+i]
    for j in range(n2):
        R[j]=array[m+1+j]

    i = j = 0
    k = l
```

Figure 3: Merge Function

```
while i<n1 and j<n2:
    if L[i]<R[j]:
        array[k]=L[i]
        i+=1
    else:
        array[k]=R[j]
        j+=1

    k+=1

while i<n1:
    array[k]=L[i]
    i+=1
    k+=1

while j<n2:
    array[k]=R[j]
    j+=1
    k+=1
```

Figure 4: Merge Comparing

```python
def mergesort(array,l,r):
    if l<r:
        m = l + (r-l)//2



        mergesort(array,l,m)
        mergesort(array,m+1,r)

        merge(array,l,m,r)
```

Figure 5: Merge Sort Recursive Function

Complexity

- **Time Complexity:** $O(n \log n)$ across Best, Average, and Worst cases.
- **Space Complexity:** $O(n)$ due to the requirement for an auxiliary array during the merge step.

4. Merge Sort Improved

Overview

This variant is an "adaptive" version of the standard Merge Sort. It is designed to recognize when the data is already partially or fully sorted. By adding a simple logical check, the algorithm can bypass the most expensive part of the process—the merging phase—whenever the data allows it.

How It Works

The "Improved" version functions exactly like standard Merge Sort until the point where two sorted sub-arrays are about to be merged. It introduces a check: if the largest element of the left sub-array is less than or equal to the smallest element of the right sub-array, it knows the combined sequence is already sorted. In this case, it skips the ‘merge()‘ function entirely. This optimization is particularly powerful for pre-sorted arrays or data with large sorted "blocks," where it can approach linear $O(n)$ performance.

```python
def mergesort(array,l,r):
    if l<r:

        m = l+(r-l)//2

        mergesort(array,l,m)
        mergesort(array,m+1,r)

        if array[m]<=array[m+1]:
            return

        merge(array,l,m,r)
```

Figure 6: Merge Sort Improved Implementation

Complexity

- **Best Case:** $O(n)$ for perfectly sorted data.
- **Average/Worst Case:** $O(n \log n)$, maintaining the same upper bound as standard Merge Sort.
- **Space Complexity:** $O(n)$.

5. Heap Sort

Overview

Heap Sort is an in-place sorting algorithm that uses a binary heap data structure. It combines the $O(n \log n)$ guarantee of Merge Sort with the $O(1)$ space efficiency of an in-place sort. While it is often slightly slower than QuickSort in practice due to poor cache locality, its predictable performance makes it a reliable choice for memory-constrained systems.

How It Works

The process begins by transforming the unsorted array into a "Max-Heap"—a binary tree where every parent node is larger than its children. Once the heap is built, the largest element is at the root. The algorithm repeatedly swaps this root with the last element of the array, effectively "extracting" the maximum. After each swap, the heap size is reduced, and the remaining tree is "heapified" to restore the max-heap property. This continues until the heap is empty and the array is fully sorted.

```python
def heapify(array,n,i):
    l = 2*i+1
    r = 2*i+2

    largest=i

    if l<n and array[l]>array[largest]:
        largest=l

    if r<n and array[r]>array[largest]:
        largest=r

    if largest!=i:
        array[i],array[largest] = array[largest],array[i]
        heapify(array,n,largest)


def heap_sort(array):
    n = len(array)

    for i in range(n//2-1,-1,-1):
        heapify(array,n,i)

    for i in range(n-1,0,-1):
        array[0],array[i] = array[i],array[0]
        heapify(array,i,0)
```

Figure 7: Heap Sort Implementation

Complexity

- **Time Complexity:** $O(n \log n)$ for Best, Average, and Worst cases.
- **Space Complexity:** $O(1)$, as it sorts the data directly within the original array.

6. Heap Sort Floyd's Bottom-Up Improvement

Overview

Floyd's Bottom-Up optimization is a mathematical refinement of the standard Heap Sort's 'sift-down' operation. It is based on the observation that most elements sifted down in a heap end up near the leaf level anyway. By reducing the number of comparisons required during this process, it significantly speeds up the extraction phase.

How It Works

In standard Heap Sort, every step down the tree requires two comparisons (to find the larger child and to check if the parent should swap). Floyd's version sifts the element all the way down to a leaf position using only one comparison per level (finding the largest child path). Once it reaches the bottom, it "sifts back up" to find its correct position. Since the path up is usually very short, this two-phase approach (fast down, short up) saves approximately 33% of the total comparisons compared to the traditional method.

```python
def heapify(array,n,i):
    start = i

    while True:
        child = 2*i+1

        if child>=n:
            break

        if child+1<n and array[child]<array[child+1]:
            child+=1

        array[child],array[i] = array[i],array[child]

        i=child

    while i>start:
        parent = (i-1)//2

        if array[i]>array[parent]:
            array[i],array[parent] = array[parent],array[i]
            i = parent
        else:
            break
```

Figure 8: Heap Sort Floyd's Bottom-Up Implementation

Complexity

- **Time Complexity:** $O(n \log n)$ with a lower constant factor than standard Heap Sort.

- **Space Complexity:** $O(1)$.

7. Persian Sort

Overview

Persian Sort is a highly advanced hybrid sorting algorithm that prioritizes maximum adaptivity by leveraging the existing structure within a dataset. Similar in philosophy to Timsort, it actively scans for "natural runs"—sequences that are already sorted in ascending or descending order. By identifying these "knots" and applying a unique "warping" logic for data movement, it achieves exceptional efficiency on real-world datasets that often contain partial ordering.

How It Works

The algorithm begins by scanning the data to identify ascending or descending runs. Descending runs are reversed in-place to become ascending. It then utilizes a "warping factor" (wp) to divide the array into segments and uses a specialized `jump_insert` function to merge them. This function moves elements in large "jumps" rather than single-element shifts, which dramatically reduces the number of memory writes and comparisons. The algorithm is recursive and adjusts its warping parameters based on the size and structure of the remaining unsorted segments, allowing it to "warp" through the data with high efficiency. For k-nearly sorted data, the `max-jump` parameter is dynamically tuned to k to further enhance performance.

```python
def jumpinsert(ar, start, i, aux, aux_start, j):
    jump = j - aux_start + 1

    while j >= aux_start:
        temp = aux[j]
        while i >= start and ar[i] > temp:
            ar[i + jump] = ar[i]
            i -= 1

        ar[i + jump] = temp
        jump -= 1
        j -= 1

def auxiliary_size(n, wp):
    m = n // wp
    if m == 0:
        return 2
    else:
        return 1 + m + auxiliary_size(m, wp)

def reverse_segment(ar, s, e):
    while s < e:
        ar[s], ar[e] = ar[e], ar[s]
        s += 1
        e -= 1
```

Figure 9: Jump Insert function

```python
def warping(ar, aux, start, end, aux_size, wp):
    if start >= end:
        return

    max_jump = ((end - start + 1) // wp) + 1
    i = start

    while i < end and ar[i + 1] < ar[i]:
        i += 1
    if i > start:
        reverse_segment(ar, start, i)

    while i < end:
        while i < end and ar[i + 1] >= ar[i]:
            i += 1

        if i == end:
            return

        if (i + max_jump) > end:
            max_jump = end - i

        if (end - start + 1) > aux_size:
            for j in range(1, max_jump + 1):
                aux[j] = ar[i + j]

            if max_jump > 1:
                warping(aux, aux, 1, max_jump, aux_size, wp)

            jumpinsert(ar, start, i, aux, 1, max_jump)
            i = i + max_jump
```

Figure 10: Warping Function 1

```
else:
    temp = end - i
    low = end + 1
    high = end + max_jump

    if high >= len(aux):
        aux.extend([0] * (high - len(aux) + 1))

    for j in range(low, high + 1):
        aux[j] = ar[j - temp]

    if max_jump > 1:
        warping(aux, aux, low, high, aux_size, wp)

    jumpinsert(ar, start, i, aux, low, high)
    i = i + max_jump
```

Figure 11: Warping Function 2

```
def persiansort(ar, wp=9):
    n = len(ar)
    if n <= 1: return ar

    start = 0
    end = n - 1
    aux_s = auxiliary_size(n, wp)

    if (end - start + 1) < wp:
        aux_s = 1

    aux = [0] * (aux_s + 1)
    warping(ar, aux, start, end, aux_s, wp)
    return ar
```

Figure 12: Persian Sort Main Function

Complexity

- **Best Case Time Complexity:** $O(n)$, which occurs when the input data is already sorted or nearly sorted, allowing the "knots" (runs) to be processed linearly.
- **Worst Case Time Complexity:** $O(n \log n)$, arising when the data is completely random and the performance of knots operations is at its minimum.

- **Space Complexity:** The memory requirement is determined by the warping factor (wp). It is defined by the series $\frac{n}{wp} + \frac{n}{wp^2} + \frac{n}{wp^3} + \ldots$, which is significantly less than $O(n)$. With the chosen optimal value of $wp = 9$, the algorithm maintains high performance stability while drastically reducing auxiliary memory consumption.

DETAILED EMPIRICAL RESULTS

The following subsections present the timing results and analysis graphs obtained from testing all seven algorithms across the full set of input distributions. Each scenario draws on timing plots from the `time` folder and summary graphs from the corresponding subfolder within `graphs`.

I. Unsorted Arrays

Big Numbers, Big Range ($N \leq 10^6$, Values $\leq 10^9$)

In the analysis of large, randomly distributed arrays with a wide value range, we observe a clear hierarchy of efficiency. **Heap Sort V2 (Floyd)** consistently outperforms V1; this is attributed to the fact that V2 utilizes a bottom-up heap construction which mathematically requires significantly fewer comparisons than the top-down insertion method found in V1.

Regarding the Merge family, **Merge Sort V2** demonstrates a speed advantage over V1 by implementing a conditional check that verifies if the last element of the left sub-array is smaller than or equal to the first element of the right sub-array. If this condition is met, the merge logic is bypassed. Interestingly, **Persian Sort**—an advanced evolution of Merge Sort—surpasses both Merge variants. Its superiority stems from its ability to identify and exploit both ascending and descending "runs," reversing the latter to create longer sorted sequences, and utilizing the `jump_insert` function which facilitates faster data movement than standard insertion.

Despite these optimizations, **QuickSort** remains the overall leader. Specifically, **QuickSort Hoare** proves to be the fastest of all tested algorithms. The results indicate that the Hoare partitioning scheme performs fewer comparisons and swaps than the Lomuto variant, making it the most efficient option for truly random data distributions.

TESTING: UNSORTED / BIG_NUMBERS

N Size	Quick V1 (Lom)	Quick V2 (Hoa)	Merge V1 (Bas)	Merge V2 (Ada)	Persian Sort	Heap V1 (Rec)	Heap V2 (Flo)
100	0.00005	0.00007	0.00014	0.00012	0.00012	0.00013	0.00011
1000	0.00084	0.00087	0.00189	0.00181	0.00137	0.00194	0.00152
100000	0.14000	0.14140	0.28572	0.26132	0.26062	0.35998	0.26046
200000	0.32736	0.28367	0.58605	0.55750	0.50046	0.70245	0.57231
400000	0.68678	0.68233	1.28348	1.25153	1.14642	1.59774	1.31147
600000	1.07650	0.96076	2.07136	1.97108	1.82284	2.73113	2.11214
800000	1.52916	1.36816	2.87738	3.02448	2.78759	4.14978	3.16960
1000000	2.01838	1.81543	3.69512	3.60540	3.30897	5.14148	3.98261

Figure 13: Timing results for unsorted arrays with big numbers and big range

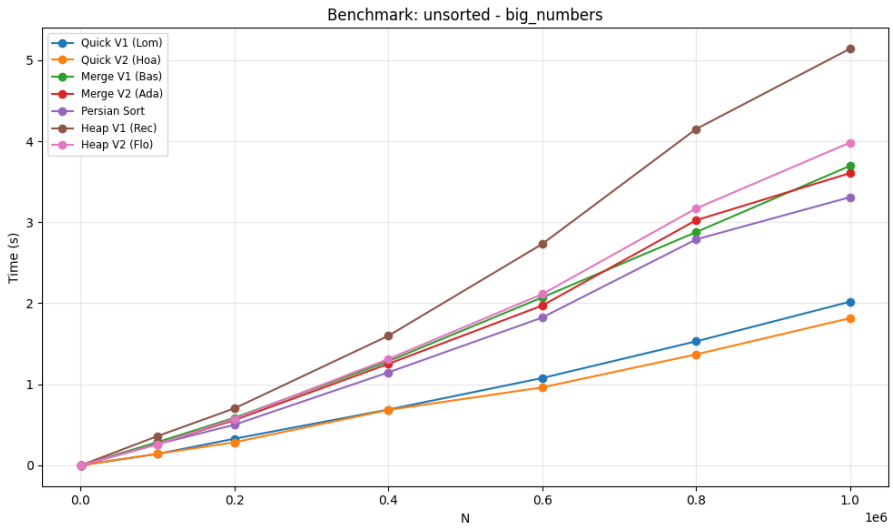

Figure 14: Graph summary for unsorted/big numbers

Small Numbers, Stress Test ($N \leq 10^6$, Values ≤ 1000)

Reducing the value range while maintaining a high N introduces a unique challenge: heavy duplicate density. In this scenario, **QuickSort Lomuto** experiences a catastrophic performance collapse, with execution time increasing quadratically ($O(N^2)$). This occurs because, with values capped at 1,000, each pivot encounters approximately 1,000 duplicates, leading to a severely unbalanced recursion tree that the Lomuto scheme cannot handle efficiently.

While **Merge Sort V2** maintains a slight edge over V1, and **Persian Sort** remains faster than both due to its run-detection, **QuickSort Hoare** continues to be the best overall performer. Its stability in this high-duplicate environment is due to its partitioning logic, which successfully distributes elements equal to the pivot to both sides of the array, preventing the tree skewing that cripples Lomuto.

```
============================================ TESTING: UNSORTED / SMALL_NUMBERS_BIG_RANGE_1 ============================================
N Size      | Quick V1 (Lom)  | Quick V2 (Hoa)  | Merge V1 (Bas)  | Merge V2 (Ada)  | Persian Sort    | Heap V1 (Rec)   | Heap V2 (Flo)   |
-----------------------------------------------------------------------------------------------------------------------------------------------
100         | 0.00013         | 0.00008         | 0.00016         | 0.00014         | 0.00012         | 0.00016         | 0.00011         |
1000        | 0.00089         | 0.00091         | 0.00198         | 0.00183         | 0.00164         | 0.00216         | 0.00163         |
100000      | 0.30759         | 0.12922         | 0.31071         | 0.28054         | 0.33574         | 0.34827         | 0.26051         |
200000      | 0.97034         | 0.31446         | 0.61307         | 0.59880         | 0.54293         | 0.73304         | 0.57325         |
400000      | 3.32143         | 0.59965         | 1.29510         | 1.24727         | 1.21380         | 1.62185         | 1.23603         |
600000      | 7.01886         | 0.93825         | 2.03954         | 1.92799         | 1.96240         | 2.81671         | 2.05089         |
800000      | 12.15666        | 1.30041         | 2.77773         | 2.68508         | 2.53525         | 3.57377         | 2.79253         |
1000000     | 18.60492        | 1.64845         | 3.54923         | 3.40025         | 3.19646         | 5.04901         | 3.59597         |
```

Figure 15: Timing plot for unsorted arrays with small numbers (dataset 1)

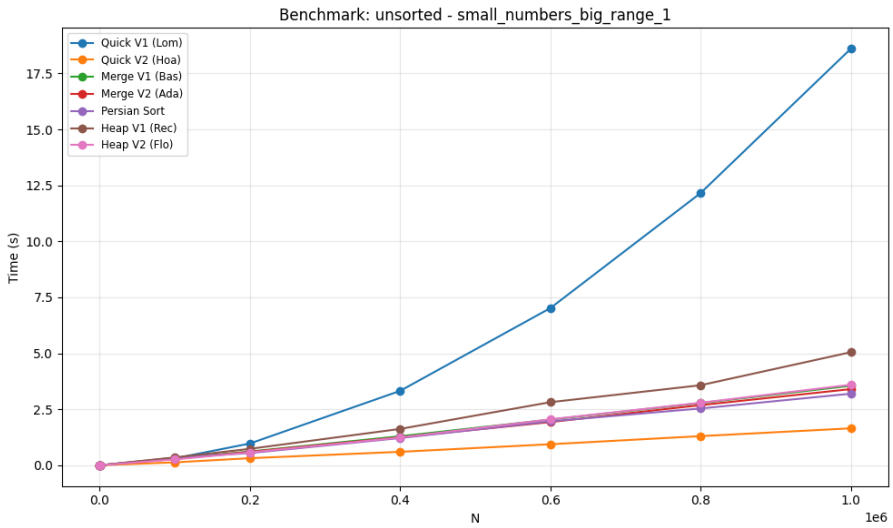

Figure 16: Graph summary for unsorted small numbers dataset 1

TESTING: UNSORTED / SMALL_NUMBERS_BIG_RANGE_2

N Size	Quick V1 (Lom)	Quick V2 (Hoa)	Merge V1 (Bas)	Merge V2 (Ada)	Persian Sort	Heap V1 (Rec)	Heap V2 (Flo)
100	SKIPPED	0.00011	0.00016	0.00014	0.00013	0.00014	0.00012
1000	SKIPPED	0.00093	0.00200	0.00190	0.00189	0.00206	0.00164
100000	SKIPPED	0.14749	0.30786	0.28057	0.26012	0.38826	0.27431
200000	SKIPPED	0.30720	0.70434	0.63956	0.56439	0.81368	0.64873
400000	SKIPPED	0.65490	1.43143	1.37015	1.24953	1.73438	1.36973
600000	SKIPPED	1.07362	2.20249	2.14227	1.81419	2.69096	2.14961
800000	SKIPPED	1.30718	2.77973	2.67092	2.50105	3.53917	2.82929
1000000	SKIPPED	1.78644	3.86789	3.74744	3.40352	4.82833	3.88660

Figure 17: Timing plot for unsorted arrays with small numbers (dataset 2)

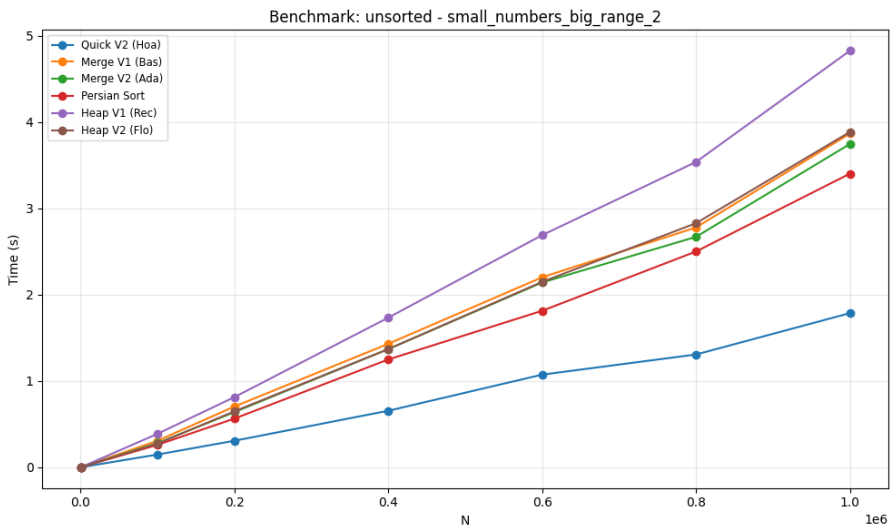

Figure 18: Graph summary for unsorted small numbers dataset 2

II. Sorted Arrays

Small Range ($N \leq 20{,}000$)

On small-scale sorted arrays, we observe the immediate impact of algorithmic "blindness." **QuickSort Lomuto** hits its absolute worst-case scenario; because it consistently selects

the last element as a pivot in a sorted sequence, it creates the most unbalanced partitions possible, leading to a visible quadratic ($O(N^2)$) growth. In stark contrast, the remaining algorithms—all of which maintain $O(N \log N)$ or better—perform so efficiently at this scale that their timing curves appear as a nearly flat line at the bottom of the graph, dwarfed by Lomuto's steep incline.

TESTING: SORTED / SMALL_RANGE

N Size	Quick V1 (Lom)	Quick V2 (Hoa)	Merge V1 (Bas)	Merge V2 (Ada)	Persian Sort	Heap V1 (Rec)	Heap V2 (Flo)
100	0.00043	0.00006	0.00014	0.00002	0.00001	0.00014	0.00011
200	0.00149	0.00011	0.00029	0.00004	0.00002	0.00032	0.00024
500	0.00963	0.00038	0.00084	0.00009	0.00004	0.00098	0.00072
1000	0.04123	0.00067	0.00199	0.00018	0.00008	0.00224	0.00166
2000	0.15985	0.00150	0.00374	0.00034	0.00014	0.00468	0.00348
5000	1.05522	0.00423	0.01226	0.00096	0.00042	0.01458	0.01141
10000	4.18803	0.00766	0.02271	0.00180	0.00074	0.02799	0.02157
20000	16.02779	0.01580	0.04600	0.00343	0.00147	0.06018	0.04759

Figure 19: Timed results for sorted arrays, small range

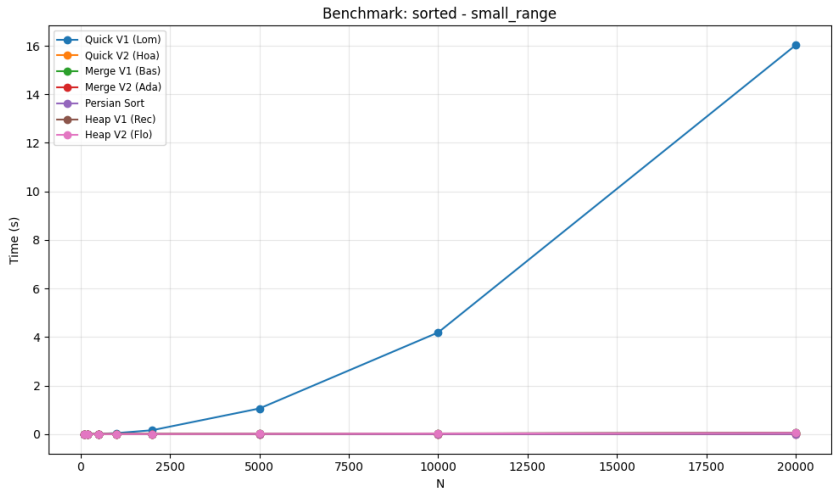

Figure 20: Graph summary for sorted small range

Big Range ($N \leq 10^6$)

As the dataset scales to one million elements, the distinction between adaptive and non-adaptive logic becomes the dominant narrative. We observe that both versions of **Heap Sort** (V1 and Floyd's V2) behave identically on sorted data as they do on unsorted data; they are entirely non-adaptive and perform the same number of operations regardless of existing order. The same consistency is seen in **Merge Sort V1**.

However, the performance of the optimized variants is a complete departure from the random baseline. **Merge Sort V2** becomes exceptionally fast, effectively approaching linear $O(N)$ complexity because its skip-merge comparison is satisfied at every recursive level. **Persian Sort** emerges as the fastest of all tested algorithms; it identifies the entire array as one continuous ascending run and concludes the process with minimal additional work. Notably, **QuickSort Hoare** performs even faster on this sorted big range than it does on unsorted data. This is likely due to the predictable partitioning pattern which

allows for near-perfect branch prediction by the CPU. We conclude that for pre-sorted datasets, **Persian Sort** and **Merge Sort V2** are the elite choices.

TESTING: SORTED / BIG_RANGE

N Size	Quick V1 (Lom)	Quick V2 (Hoa)	Merge V1 (Bas)	Merge V2 (Ada)	Persian Sort	Heap V1 (Rec)	Heap V2 (Flo)
100	0.00040	0.00005	0.00014	0.00002	0.00001	0.00014	0.00011
1000	0.04007	0.00066	0.00183	0.00018	0.00008	0.00221	0.00164
100000	SKIPPED	0.09955	0.26670	0.01699	0.00698	0.35683	0.27581
200000	SKIPPED	0.24130	0.63551	0.03836	0.01670	0.78271	0.59502
400000	SKIPPED	0.45742	1.27302	0.06874	0.03037	1.64830	1.26821
600000	SKIPPED	0.62596	2.06806	0.11116	0.05312	2.69588	1.90614
800000	SKIPPED	0.79218	2.44008	0.13276	0.05790	3.30977	2.53169
1000000	SKIPPED	1.10804	3.26804	0.16828	0.07273	4.14525	3.20949

Figure 21: Timed results for sorted arrays, big range

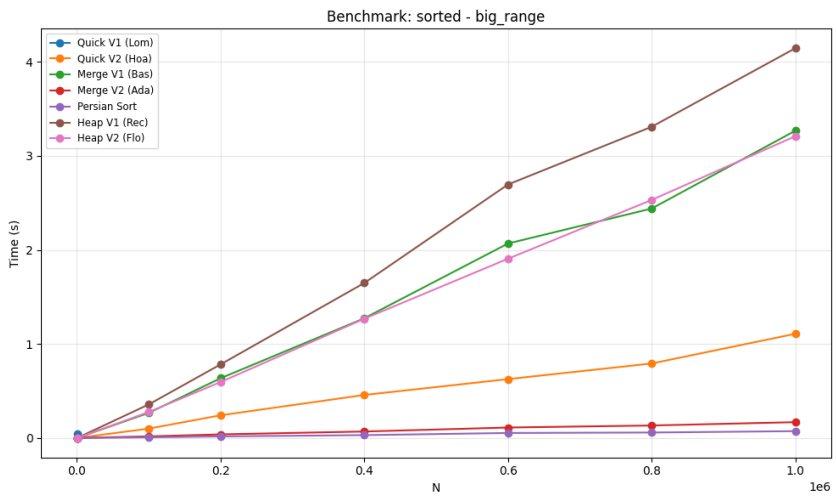

Figure 22: Graph summary for sorted big range

III. Sorted Descending Arrays

Small Range ($N \leq 20000$)

On small-scale reverse-sorted arrays, **QuickSort Lomuto** continues to exhibit $O(N^2)$ behavior, similar to the ascending-sorted case. However, we observe a subtle but consistent improvement in execution time compared to the ascending-sorted benchmark. This is due to the mechanical cost of swaps: in a sorted array, Lomuto's comparison results in a swap for nearly every element, whereas in a descending array, the comparison frequently fails, allowing the algorithm to skip the swap operation and only increment the index. Since a CPU comparison is significantly cheaper than a memory swap, the descending case appears "faster" despite the identical complexity. As seen previously, the remaining algorithms remain tightly clustered at the bottom of the temporal axis.

TESTING: SORTED_DESC / SMALL_RANGE

N Size	Quick V1 (Lom)	Quick V2 (Hoa)	Merge V1 (Bas)	Merge V2 (Ada)	Persian Sort	Heap V1 (Rec)	Heap V2 (Flo)
100	0.00052	0.00010	0.00026	0.00027	0.00002	0.00022	0.00021
200	0.00208	0.00022	0.00050	0.00062	0.00005	0.00055	0.00051
500	0.01323	0.00069	0.00146	0.00134	0.00011	0.00121	0.00109
1000	0.04644	0.00102	0.00247	0.00277	0.00018	0.00266	0.00227
2000	0.14810	0.00189	0.00512	0.00477	0.00027	0.00498	0.00480
5000	0.87796	0.00563	0.01696	0.01504	0.00077	0.01597	0.01308
10000	3.35282	0.00838	0.02269	0.02380	0.00117	0.02788	0.02274
20000	11.51513	0.01672	0.04588	0.04752	0.00218	0.05459	0.04420

Figure 23: Timing for sorted-desc arrays, small range

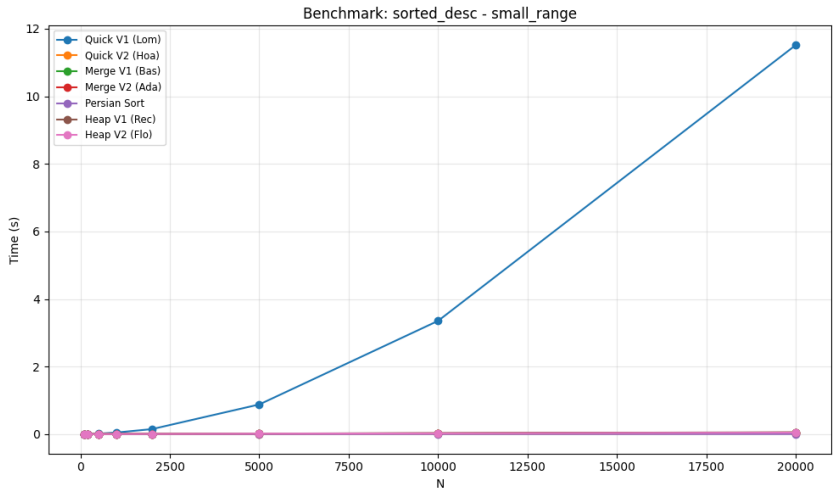

Figure 24: Graph summary for sorted-desc small range

Big Range ($N \leq 10^6$)

For large-scale reverse-sorted data, the consistency of non-adaptive algorithms remains evident. Both versions of **Heap Sort** and **Merge Sort V1** perform almost exactly as they did on ascending-sorted and random datasets, as their logic does not account for existing data patterns.

A significant performance shift is observed in **Merge Sort V2**. Unlike the ascending-sorted case where it approached linear time, here it becomes noticeably slower. Because the sub-arrays in a descending distribution are never in relative order, the skip-merge check (`array[m] <= array[m+1]`) fails at every recursive level, forcing the algorithm to perform full merges. **Persian Sort** also requires more time here than on ascending data because it must spend computational resources to identify and reverse the descending runs before it can begin the merging process.

Ultimately, **QuickSort Hoare** wins this test by a narrow margin. Its symmetric two-pointer partitioning scheme is naturally efficient at handling reversed data, moving elements from the extreme ends to their correct halves with minimal operations. **Persian Sort** follows closely as the second fastest, proving its robustness even when data must be pre-processed (reversed) before sorting.

TESTING: SORTED_DESC / BIG_RANGE

N Size	Quick V1 (Lom)	Quick V2 (Hoa)	Merge V1 (Bas)	Merge V2 (Ada)	Persian Sort	Heap V1 (Rec)	Heap V2 (Flo)
100	0.00030	0.00005	0.00014	0.00014	0.00002	0.00012	0.00011
1000	0.03010	0.00069	0.00182	0.00197	0.00015	0.00196	0.00170
100000	SKIPPED	0.09881	0.26610	0.27852	0.06802	0.33849	0.33634
200000	SKIPPED	0.20027	0.61840	0.57884	0.17480	0.70793	0.64748
400000	SKIPPED	0.43174	1.27773	1.28236	0.58470	1.55124	1.25545
600000	SKIPPED	0.62761	1.83300	1.87034	0.87289	2.24542	1.90585
800000	SKIPPED	0.81188	2.43338	2.46648	1.23179	3.19816	2.64661
1000000	SKIPPED	1.02922	3.05211	3.13072	1.66731	3.90435	3.42165

Figure 25: Timing for sorted-desc arrays, big range

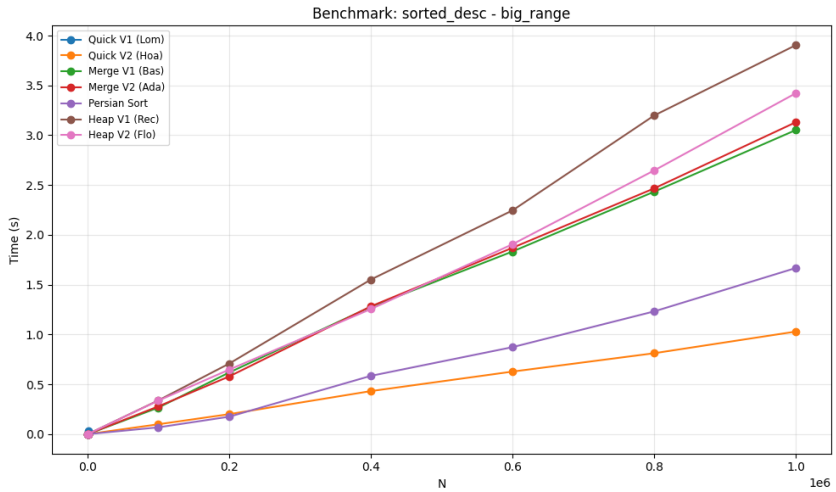

Figure 26: Graph summary for sorted-desc big range

IV. Duplicate Arrays (All elements same)

Small Range

When all elements in the array are identical, **QuickSort Lomuto** operates at its absolute theoretical and practical worst. Because the pivot is always equal to every other element, the partition fails to split the array into two halves; instead, it peels off exactly one element per recursive call. This leads to a maximum recursion depth of n and an $O(N^2)$ time complexity that dominates the graph. The remaining algorithms, being significantly more efficient at this scale, appear as a nearly flat line by comparison.

```
================================================ TESTING: DUPLICATES / SMALL_RANGE ================================================
N Size      | Quick V1 (Lom)  | Quick V2 (Hoa)  | Merge V1 (Bas)  | Merge V2 (Ada)  | Persian Sort    | Heap V1 (Rec)   | Heap V2 (Flo)
-----------------------------------------------------------------------------------------------------------------------------------
100         | 0.00024         | 0.00007         | 0.00015         | 0.00002         | 0.00001         | 0.00004         | 0.00011         |
200         | 0.00063         | 0.00014         | 0.00029         | 0.00004         | 0.00002         | 0.00006         | 0.00024         |
500         | 0.00490         | 0.00042         | 0.00085         | 0.00009         | 0.00004         | 0.00016         | 0.00070         |
1000        | 0.02115         | 0.00109         | 0.00217         | 0.00020         | 0.00009         | 0.00038         | 0.00189         |
2000        | 0.09165         | 0.00238         | 0.00515         | 0.00044         | 0.00020         | 0.00091         | 0.00435         |
5000        | 0.45449         | 0.00570         | 0.01302         | 0.00105         | 0.00041         | 0.00207         | 0.01150         |
10000       | 1.99159         | 0.01054         | 0.02215         | 0.00170         | 0.00070         | 0.00301         | 0.02098         |
20000       | 6.93626         | 0.02195         | 0.04579         | 0.00350         | 0.00138         | 0.00606         | 0.04221         |
```

Figure 27: Timing for duplicate arrays, small range

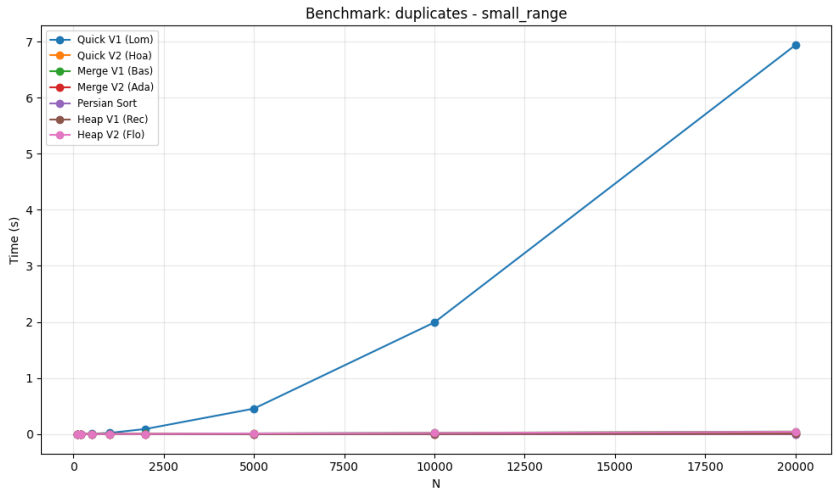

Figure 28: Graph summary for duplicates small range

Big Range ($N \leq 10^6$)

At the scale of one million elements, we observe several unique performance shifts. **Merge Sort V1**, **Merge Sort V2**, and **Heap Sort V2** all behave consistently with their results from the sorted datasets. However, **QuickSort Hoare** requires significantly more time here than it did on the sorted big range. This is due to the nature of its partitioning condition: because every element is equal to the pivot, the pointers (`while array[i] < pivot`) do not stop advancing at out-of-place elements, leading to a massive amount of redundant swaps that do not change the array state but consume many CPU cycles.

The most remarkable result in this category belongs to **Heap Sort V1**. It emerges as the fastest algorithm on this distribution, outperforming even the Floyd-optimized V2 and the Merge variants. The reason lies in an "Early Exit" implementation detail: during the heapify process, V1 checks `if largest != i` before executing a swap. In an array of identical elements, the parent is always equal to its children, so this condition is never satisfied. Consequently, V1 performs zero swaps, exiting the heapify calls immediately. While Floyd's V2 blindly sifts down to the leaf level, V1's simple logical guard allows it to bypass the work entirely. We conclude that for datasets with high duplicate density, **Heap Sort V1** is an exceptionally efficient choice.

TESTING: DUPLICATES / BIG_RANGE

N Size	Quick V1 (Lom)	Quick V2 (Hoa)	Merge V1 (Bas)	Merge V2 (Ada)	Persian Sort	Heap V1 (Rec)	Heap V2 (Flo)
100	0.00024	0.00007	0.00014	0.00002	0.00001	0.00003	0.00011
1000	0.01734	0.00088	0.00178	0.00022	0.00015	0.00032	0.00149
100000	SKIPPED	0.13561	0.26628	0.01707	0.00688	0.03092	0.29970
200000	SKIPPED	0.26761	0.64151	0.03453	0.01450	0.06659	0.52542
400000	SKIPPED	0.55437	1.18742	0.06945	0.03017	0.12707	1.08875
600000	SKIPPED	0.85570	1.81124	0.10145	0.04563	0.18344	1.70025
800000	SKIPPED	1.18573	2.38213	0.12886	0.05360	0.24900	2.30954
1000000	SKIPPED	1.45443	3.04144	0.16331	0.07447	0.30292	3.00857

Figure 29: Timing for duplicate arrays, big range

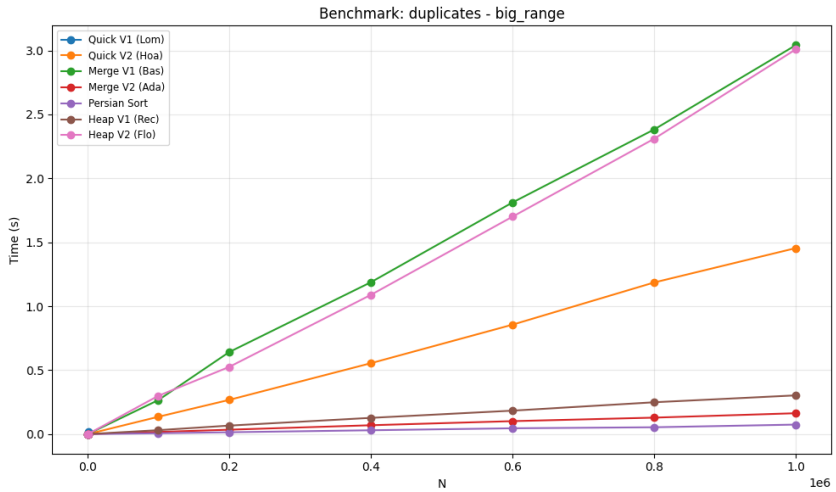

Figure 30: Graph summary for duplicates big range

V. Organ Arrays (Symmetric Hill)

Small Range

The organ-pipe distribution, characterized by a symmetric "hill" where values increase to a peak and then mirror back down, produces a catastrophic failure for **QuickSort Hoare** at small scales. In this specific scenario, we observe Hoare's execution time increasing even faster than Lomuto's. This occurs because our implementation selects the middle element as the pivot; in an organ distribution, the middle element is the peak (the maximum value). Consequently, every other element in the array is smaller than the pivot, forcing a maximally unbalanced partition. The algorithm is forced to traverse the entire array and perform a massive number of comparisons and swaps at every level of recursion. The remaining algorithms, maintaining their efficiency, appear as a nearly flat line by comparison.

TESTING: ORGAN / SMALL_RANGE

N Size	Quick V1 (Lom)	Quick V2 (Hoa)	Merge V1 (Bas)	Merge V2 (Ada)	Persian Sort	Heap V1 (Rec)	Heap V2 (Flo)
100	0.00011	0.00013	0.00015	0.00010	0.00005	0.00014	0.00012
200	0.00021	0.00046	0.00031	0.00019	0.00009	0.00032	0.00025
500	0.00177	0.00243	0.00086	0.00054	0.00020	0.00105	0.00075
1000	0.00739	0.01014	0.00200	0.00131	0.00040	0.00225	0.00168
2000	0.02815	0.04173	0.00408	0.00253	0.00076	0.00468	0.00364
5000	0.13001	0.22721	0.01170	0.00673	0.00211	0.01362	0.01022
10000	0.50408	0.98594	0.02464	0.01414	0.00393	0.02999	0.02301
20000	2.07407	3.71054	0.05055	0.02899	0.00810	0.06363	0.04753

Figure 31: Timing for organ arrays, small range

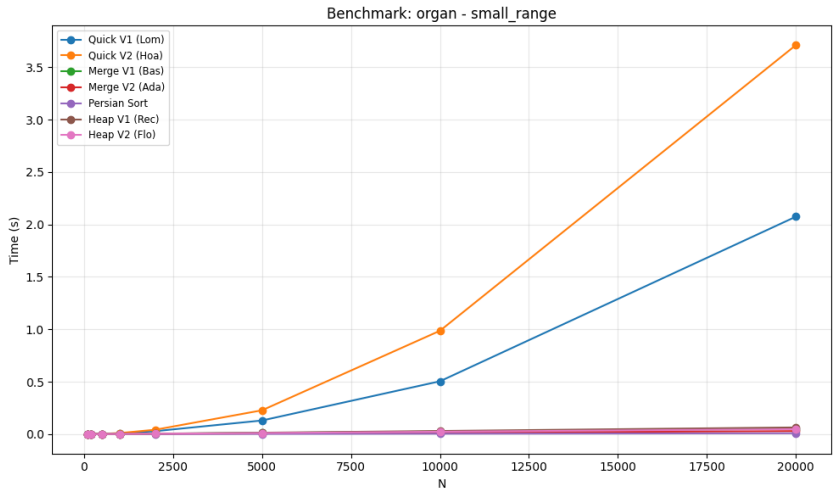

Figure 32: Graph summary for organ small range

Big Range ($N \leq 10^6$)

At full scale, the non-adaptive algorithms (**Heap Sort V1**, **V2**, and **Merge Sort V1**) continue to behave consistently with their performance on previous distributions. However, **Merge Sort V2** exhibits an interesting "Arithmetic Median" behavior. Because the left half of the hill is ascending (triggering the skip-merge optimization) and the right half is descending (forcing a full merge), its total execution time sits exactly between its best-case and worst-case results. While it is significantly faster than the standard non-adaptive algorithms, it cannot match the efficiency of the top performer.

The undisputed winner in this category is **Persian Sort**. Its adaptive logic is perfectly suited for this distribution; it identifies the ascending first half and the descending second half as two distinct runs. By immediately reversing the descending run, it converts the entire problem into a simple merge of two sorted sequences, finishing in near-linear time. **Merge Sort V2** remains slower than Persian Sort because, while it benefits from the sorted left side, it still must perform the expensive merging process for the entire right-side descending "knot."

TESTING: ORGAN / BIG_RANGE

N Size	Quick V1 (Lom)	Quick V2 (Hoa)	Merge V1 (Bas)	Merge V2 (Ada)	Persian Sort	Heap V1 (Rec)	Heap V2 (Flo)
100	0.00008	0.00014	0.00015	0.00010	0.00005	0.00014	0.00012
1000	0.00715	0.01012	0.00193	0.00116	0.00040	0.00234	0.00186
100000	SKIPPED	SKIPPED	0.28332	0.17075	0.03875	0.36835	0.30612
200000	SKIPPED	SKIPPED	0.59655	0.32922	0.09682	0.79449	0.61191
400000	SKIPPED	SKIPPED	1.35312	0.71364	0.19940	1.60571	1.28020
600000	SKIPPED	SKIPPED	1.93099	1.14357	0.26609	2.51874	1.92630
800000	SKIPPED	SKIPPED	2.49681	1.40230	0.37634	3.54106	2.67026
1000000	SKIPPED	SKIPPED	3.19991	1.82206	0.42851	4.15622	3.24234

Figure 33: Timing for organ arrays, big range

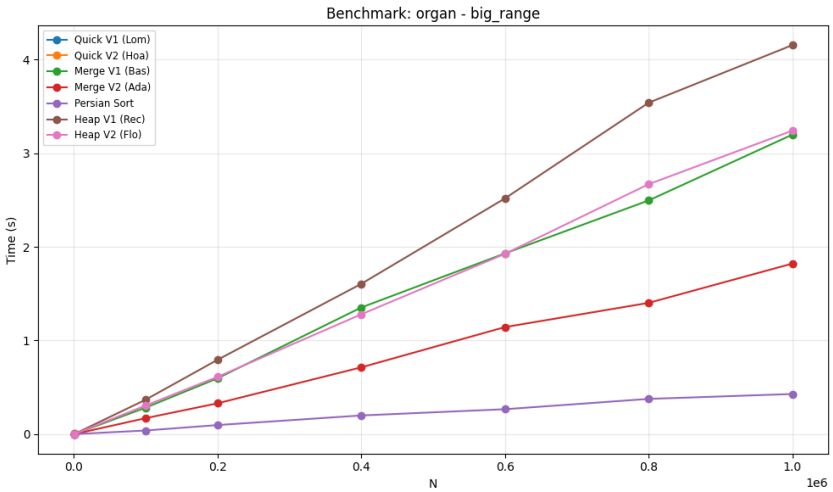

Figure 34: Graph summary for organ big range

CONCLUSIONS

This empirical analysis of seven sorting algorithms, tested across a range of input sizes and five distinct distribution patterns, has produced a clear and nuanced picture of practical sorting performance.

The most immediate takeaway is that no single algorithm is optimal for all input types. Algorithm selection must be guided by knowledge of the data. **QuickSort Hoare** proved to be the superior general-purpose choice for random datasets and even showed surprising resilience on sorted and reversed data. While it avoids many of the $O(n^2)$ pitfalls that cripple **QuickSort Lomuto**, it is not without risk; the "Organ-Pipe" distribution exposed a critical vulnerability where selecting a middle pivot (the peak) leads to a catastrophic performance collapse.

Merge Sort Improved demonstrates how a single well-placed check can transform a standard algorithm. The pre-merge comparison of adjacent boundaries adds negligible overhead on random data but allows the algorithm to approach linear time on sorted and nearly sorted inputs. This is a clear example of an optimization that costs almost nothing in the average case but pays substantial dividends when data has existing structure.

Heap Sort occupies a unique role as the most "reliable" algorithm. While it is rarely the absolute fastest, it is never catastrophically slow. Its guaranteed $O(n \log n)$ behavior and in-place operation make it the most predictable option when memory is constrained. A particularly instructive result came from the all-duplicates test, where **Heap Sort V1** outperformed every other algorithm. Thanks to its "early-exit" guard that prevents unnecessary swaps, it successfully bypassed the work that even more advanced algorithms like Floyd's V2 blindly performed.

Persian Sort emerged as the most versatile and adaptive algorithm overall. While competitive on random data, it pulls ahead significantly on structured datasets. Its run-detection logic allows it to exploit partial ordering that other algorithms ignore, and its performance on organ-pipe input—where it identified two runs and merged them in near-linear time—was the most striking result of the entire analysis.

In conclusion, this study serves as a reminder that algorithmic performance is not determined solely by asymptotic complexity. Implementation details—such as Hoare's two-pointer scan, Merge V2's skip-check, or Heap V1's early-exit guard—interact with specific data characteristics to create the final performance profile seen in our empirical results.

REFERENCES

1. **PersianSort: An Alternative to MergeSort Inspired by Persian Rug.** *ArXiv Preprint.* Available at: https://arxiv.org/html/2505.05775v4.

Access the full implementation and analysis on GitHub:

https://github.com/bogdan456tech/aa-labs/tree/main/lab2